Reimplementing LoRA: Low-Rank Adaptation of Large Language Models

Rohan Shankar[1] Harshaan Chugh[1]
[1]Cornell University, CS 4782 — Spring 2026 {rs2656,hsc53}@cornell.edu

Abstract. We reimplemented LoRA [1] from scratch and tested it on SST-2 and MRPC with BERT-base and on WikiText-2 with GPT-2 Small. LoRA hit **93.1%** validation accuracy on SST-2 and **89.9%** validation F1 on MRPC while only training 0.27% of the parameters. On WikiText-2, LoRA got 24.27 perplexity compared to 23.60 for full fine-tuning, using 99.5% fewer trainable parameters. Overall, our results back up LoRA's main claim that low-rank task updates can hold up performance while cutting way down on trainable weights.

1. Introduction

Fine-tuning large pretrained language models is expensive because you have to update every parameter and usually store a separate full model copy per task. We reproduced *LoRA: Low-Rank Adaptation of Large Language Models* by Hu et al. [1]. The paper's key observation is that downstream weight updates tend to have low intrinsic rank, so you can freeze the pretrained weights and just learn a low-rank additive update for selected matrices.

We reimplemented LoRA ourselves rather than relying on PEFT or adapter libraries. We wanted to see if the same efficiency-quality tradeoff shows up in a smaller, more reproducible setting: BERT-base [2] on GLUE tasks SST-2 and MRPC [4–6], and GPT-2 Small [3] on WikiText-2 [7].

2. Chosen Result

We tried to reproduce the paper's main empirical result: that LoRA can match or get close to full fine-tuning performance while only training a small number of low-rank parameters. This comes directly from the paper's low-rank update equation, $\Delta W = BA$, and its benchmark results showing competitive adaptation with far fewer trainable weights [1].

We picked this result because it directly tests LoRA's core claim rather than just checking implementation details. If task adaptation really is well approximated by a low-rank update, then LoRA should stay competitive with full fine-tuning, performance should go up with rank until it levels off, and the trainable parameter count should stay way smaller.

3. Methodology

3.1. LoRA Layer Design

For a frozen pretrained weight matrix $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$, LoRA learns two small matrices $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$:

$$h = W_0 x + \frac{\alpha}{r} BAx. \tag{1}$$

Here r is the LoRA rank and α/r scales the update. We initialize A with Kaiming uniform and B to zeros so that $\Delta W = 0$ at the start of training. That way the model starts out behaving exactly like the base model before any task-specific learning kicks in.

We implemented `LoRALinear` for standard `nn.Linear` modules and `LoRAConv1D` for GPT-2's `Conv1D` attention projection. Both wrappers run the frozen base path and the low-rank residual path in parallel, then add the scaled residual.

3.2. Injection, Freezing, and Models

Our `apply_lora_to_model()` function walks the model graph and swaps out modules whose names match the target suffixes. For BERT-base, we insert LoRA into the `query` and `value` projections in every transformer layer. For GPT-2 Small, we insert it into `c_attn`, GPT-2's combined attention projection. After swapping, we freeze all the pretrained parameters, so only `lora_A`, `lora_B`, and (for classification) the task head are actually trained.

3.3. Datasets, Metrics, and Training Setup

For classification, we look at SST-2 validation accuracy and MRPC validation F1. For language modeling, we use WikiText-2 validation perplexity. We sweep $r \in \{1, 4, 8, 16\}$ to see how low-rank capacity affects things. For BERT-base we target `query,value` and for GPT-2 Small we target `c_attn`. Our scripts and notebooks handle both setups and spit out the tables and figures in the report. Since we were working with limited Colab compute, this is a smaller-scale reproduction. The goal was to reproduce the qualitative trend (competitive performance with way fewer trainable parameters) rather than exactly matching every number from the original paper.

4. Results & Analysis

Table 1 summarizes the best validation metrics. On both SST-2 and MRPC, LoRA beat our full fine-tuning baselines. $r = 8$ hit 93.1% accuracy on SST-2 and 89.9% F1 on MRPC while only training 0.27%

of the parameters. That really backs up LoRA's core claim that low-rank adaptation can be very parameter-efficient.

Table 1: Best validation metrics. Parentheses show percent trainable parameters.

Task	Method	Params	Metric
SST-2	Full FT	109.5M	85.3% acc
	LoRA $r = 1$	38K (0.04%)	92.0% acc
	LoRA $r = 4$	149K (0.14%)	92.7% acc
	LoRA $r = 8$	296K (0.27%)	**93.1% acc**
	LoRA $r = 16$	591K (0.54%)	92.7% acc
MRPC	Full FT	109.5M	82.6% F1
	LoRA $r = 1$	38K (0.04%)	88.8% F1
	LoRA $r = 4$	149K (0.14%)	88.6% F1
	LoRA $r = 8$	296K (0.27%)	**89.9% F1**
	LoRA $r = 16$	591K (0.54%)	89.6% F1
Wiki-2	Full FT	124.4M	**23.60 ppl**
	LoRA $r = 1$	37K (0.03%)	24.99 ppl
	LoRA $r = 4$	147K (0.12%)	24.43 ppl
	LoRA $r = 8$	295K (0.24%)	24.34 ppl
	LoRA $r = 16$	590K (0.47%)	24.27 ppl

The biggest surprise was that our full fine-tuning baselines actually underperformed LoRA on GLUE. We think this comes down to overfitting: when you update 109.5M parameters on a small classification dataset, there's not enough regularization. LoRA sidesteps this by restricting learning to a lower-dimensional subspace, which naturally limits overfitting.

On WikiText-2, full fine-tuning did stay slightly ahead: 23.60 perplexity versus 24.27 for LoRA at $r = 16$. That makes sense since language modeling probably benefits from updating more of the model, and we only targeted `c_attn` in GPT-2. But even then the gap was pretty small given the huge difference in parameter count.

The rank sweep also backed up the low-rank hypothesis. On classification, performance goes up from $r = 1$ to $r = 8$ and then kind of levels off or even drops a bit at $r = 16$. On WikiText-2, perplexity keeps improving through $r = 16$. It looks like how much rank you need really depends on the task: small classification tasks don't need many degrees of freedom, while language modeling needs more capacity. LoRA also cut peak GPU memory by up to 58% on SST-2, which is a nice practical bonus.

5. Reflections

This project made it clear that parameter-efficient fine-tuning isn't just about the math. A lot of the work was getting the implementation details right: figuring out which modules to replace, making sure the base weights were actually frozen, initializing A and B correctly, and keeping the task head trainable.

Our results confirm LoRA's main trend: low-rank updates can get strong downstream performance with way fewer trainable parameters. If we had more time, we'd run multiple seeds, do more careful tuning on the full fine-tuning baselines, try additional insertion targets like key/output/MLP projections, and add LoRA weight merging to test inference speed directly. We'd also be curious to try rank-adaptive methods that learn a per-layer rank on their own instead of fixing r globally.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*, 2019.

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.

[4] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *ICLR*, 2019.

[5] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. Manning, A. Ng, and C. Potts. Recursive deep models for semantic compositionality over a sentiment treebank. *EMNLP*, 2013.

[6] W. Dolan and C. Brockett. Automatically constructing a corpus of sentential paraphrases. *IWP*, 2005.

[7] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. *ICLR*, 2017.

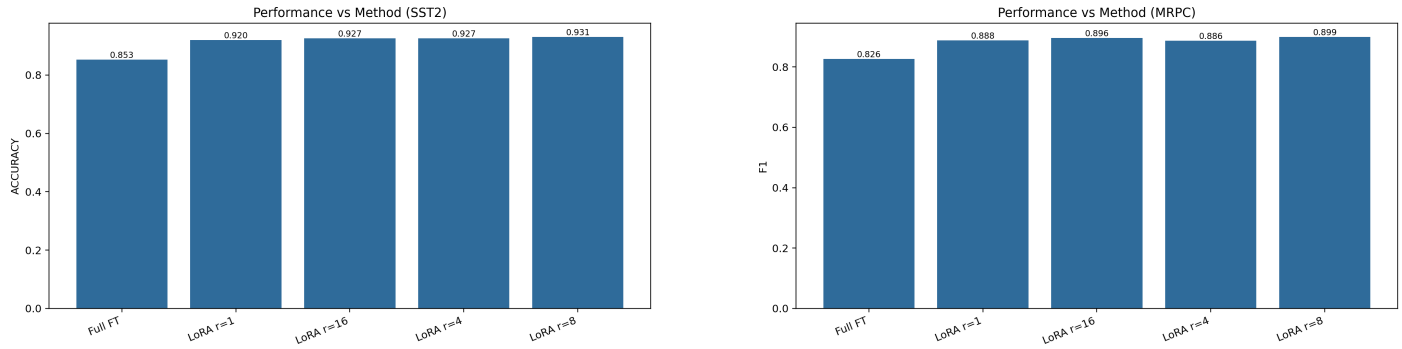
[8] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*, 2019.

[9] T. Wolf et al. Transformers: State-of-the-art natural language processing. *EMNLP: System Demonstrations*, 2020.

[10] Q. Lhoest et al. Datasets: A community library for natural language processing. *EMNLP: System Demonstrations*, 2021.

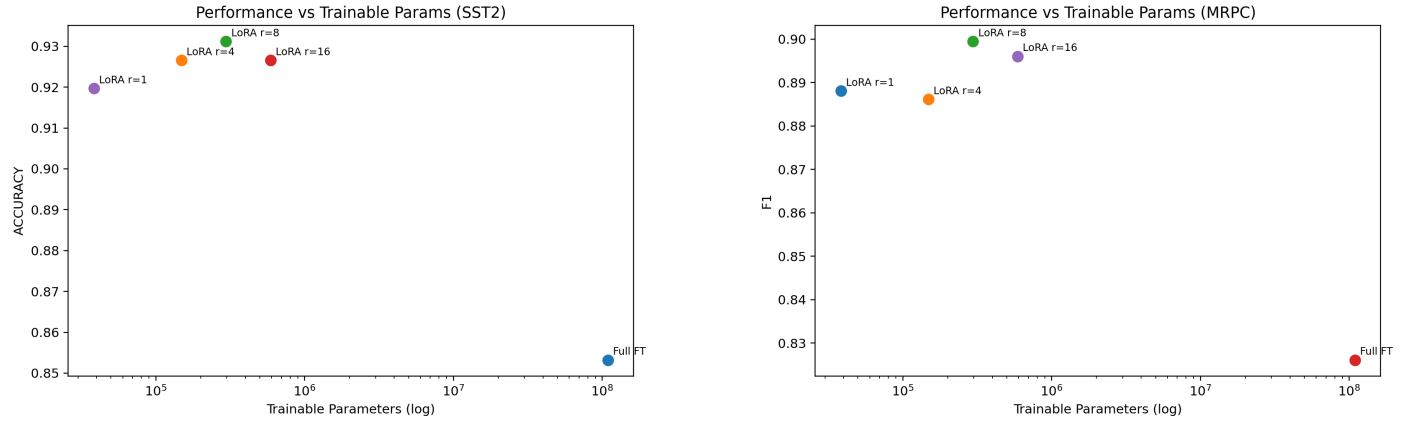
Figures

All figures are collected here after the 2-page narrative to keep the text pages clean and within the page limit (figures are excluded from the limit per the report instructions).



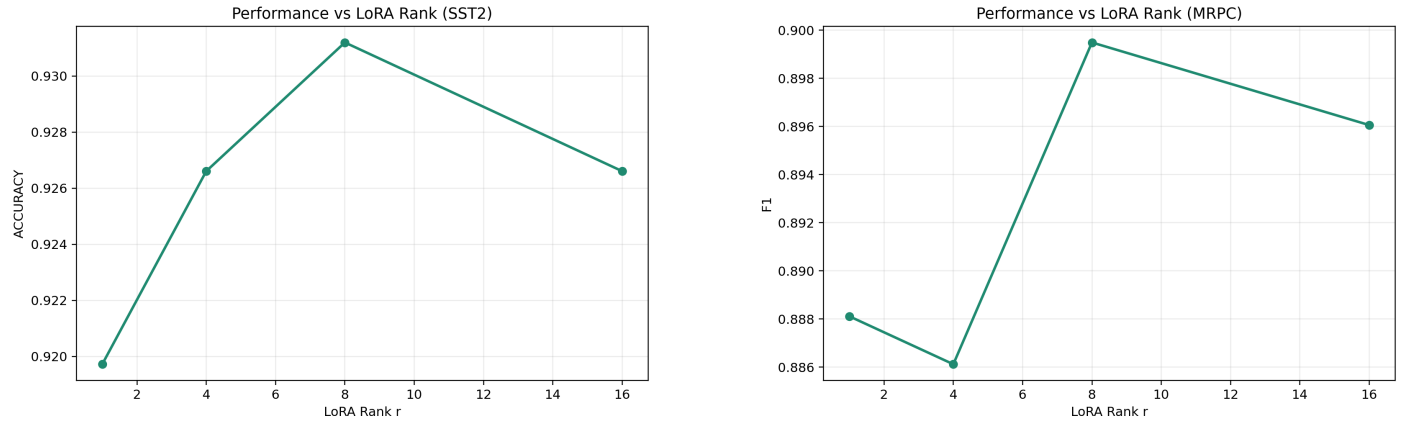
(a) SST-2 **(b)** MRPC

Figure 1: Performance comparison between full fine-tuning and LoRA ranks.



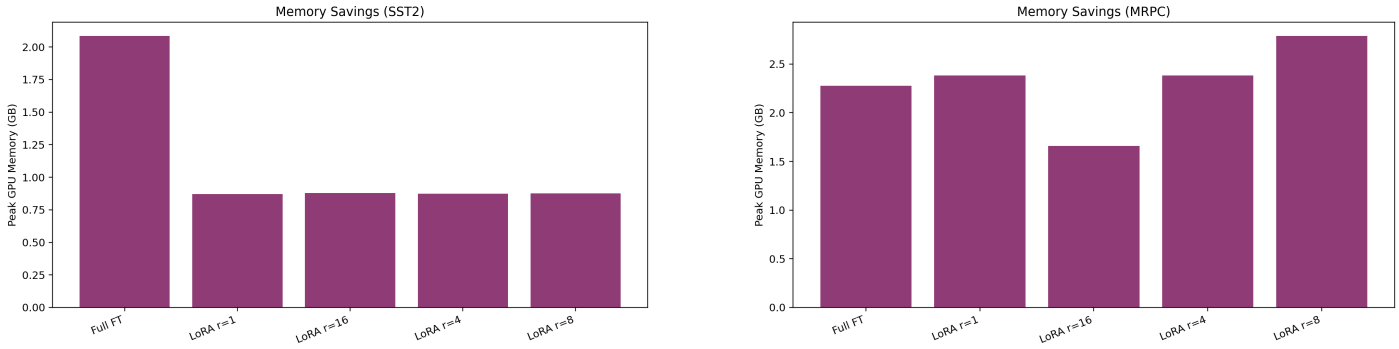
(a) SST-2 **(b)** MRPC

Figure 2: Performance versus trainable parameters. LoRA occupies the high-performance, low-parameter region.



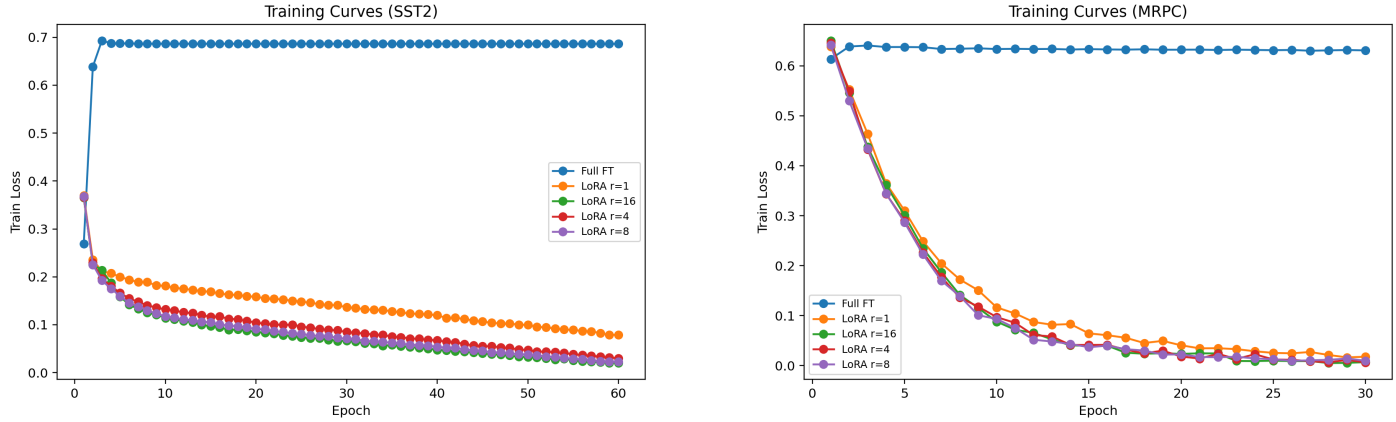
(a) SST-2 **(b)** MRPC

Figure 3: Rank ablation. Classification performance improves at low rank and saturates around $r = 8$.



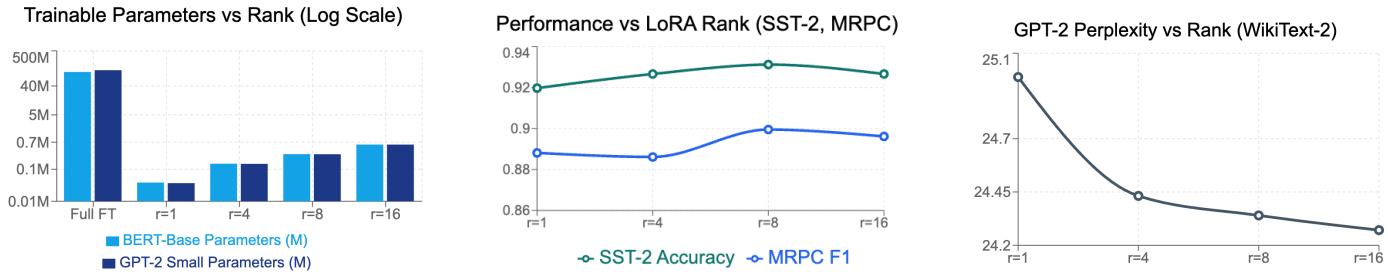
(a) SST-2 **(b)** MRPC

Figure 4: Peak GPU memory comparison. LoRA reduces the memory required for adaptation.



(a) SST-2 **(b)** MRPC

Figure 5: Training curves for full fine-tuning and LoRA variants.



(a) Trainable params vs. rank **(b)** GLUE metrics vs. rank **(c)** GPT-2 perplexity vs. rank

Figure 6: Cross-model summary of rank, parameter count, and language-modeling perplexity.